Announcements

For Project 1A (due today), there is no way to run or even compile your code locally.

- For now, you're simply writing tests and submitting them.
- Feel free to submit however many times you want, as often as you want.

For Project 1B there is a velocity limit.

- No more than 4 submissions per day. You should be using your local tests to debug, not ours.

CC BY NC SA



Lecture 7 (Lists 3)

DLLists and Basic ALists

CS61B, Fall 2026 @ UC Berkeley

Josh Hug and Manuel Sabin

CC BY NC SA

Summary of SLLists So Far

Lecture 7, CS61B, Fall 2026

SLLists:

- **Summary of SLLists So Far**
- Why a Last Pointer Isn't Enough
- Doubly Linked Lists
- Generic Lists
- Linked Lists Are Bad at Get

ALists:

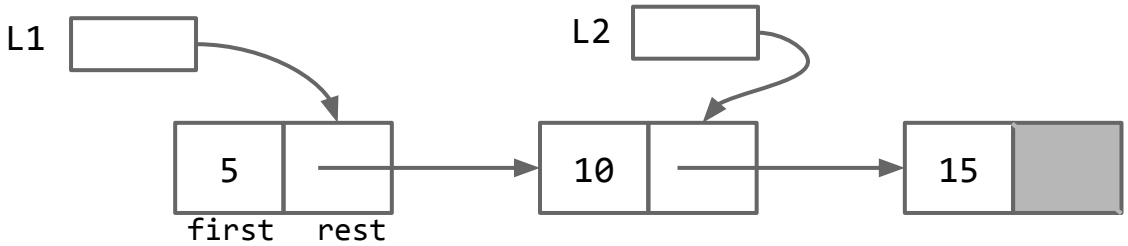
- Array Review
- Basic ArrayList Implementation
- removeLast and the Allegory of the Cave
- removeLast Implementation

CC BY NC SA

Allegory of the Cave and Sentinel

In Lecture 4, we built the `IntList`.

- Requires programmers using an `IntList` to have a strong understanding of recursion and pointers.
- A hassle to use, even for strong programmers (e.g. no `addFirst` method possible).



CC BY NC SA

Allegory of the Cave

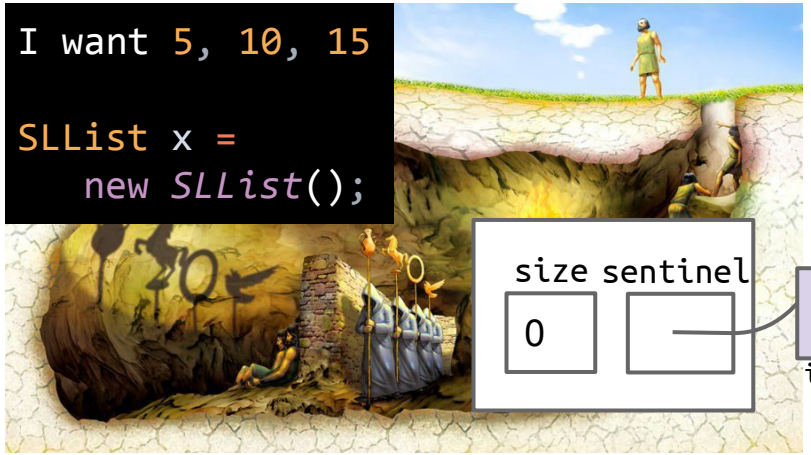


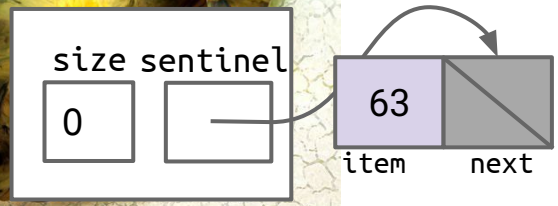
CC BY NC SA

Allegory of the Cave

```
I want 5, 10, 15

SLList x =
    new SLList();
```



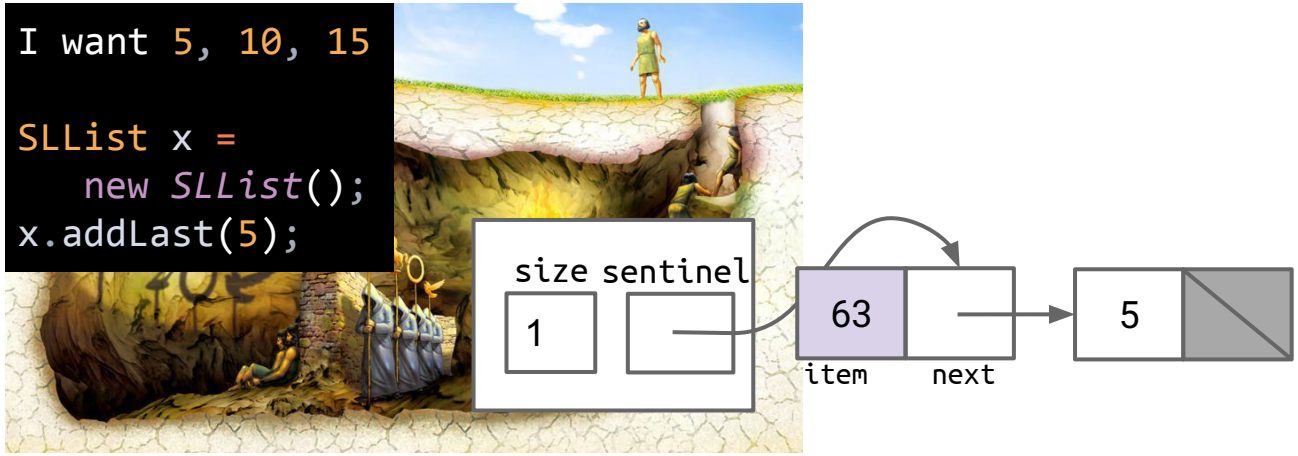


CC BY NC SA

Allegory of the Cave

```
I want 5, 10, 15

SLList x =
    new SLList();
x.addLast(5);
```

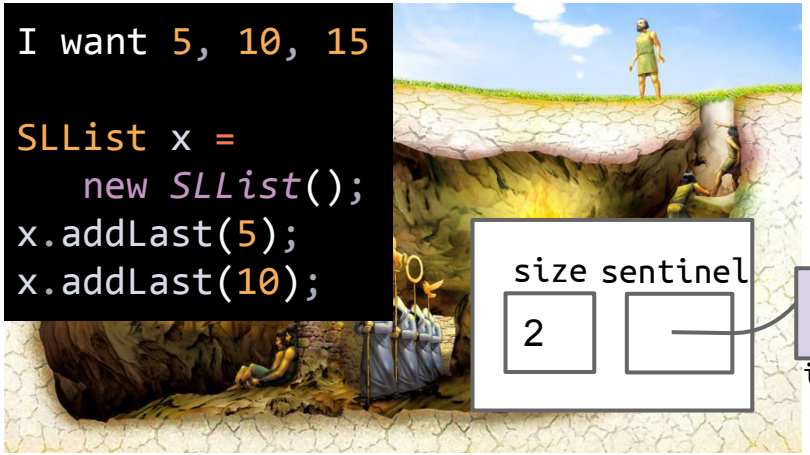


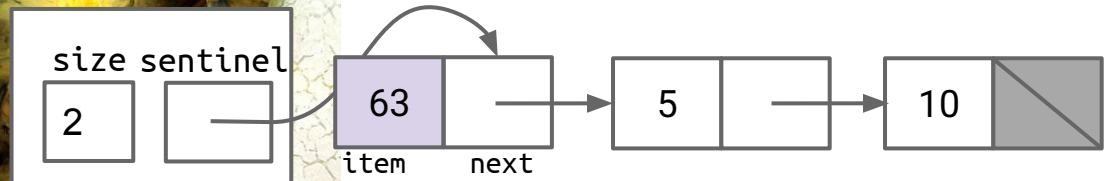
CC BY NC SA

Allegory of the Cave

```
I want 5, 10, 15

SLList x =
    new SLList();
x.addLast(5);
x.addLast(10);
```



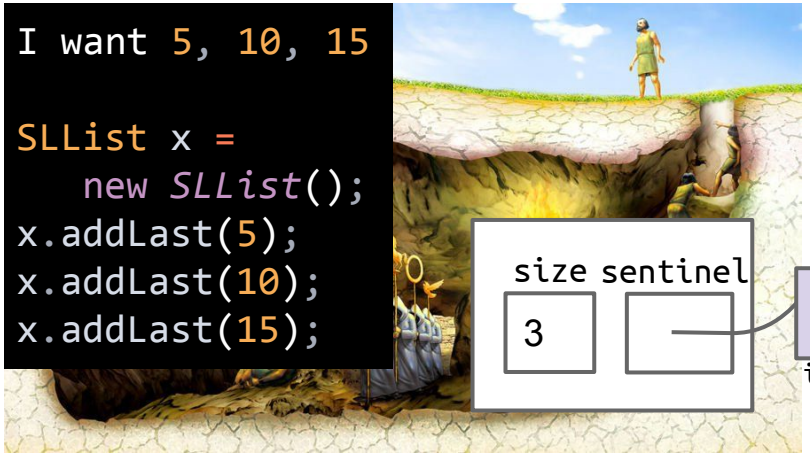


CC BY NC SA

Allegory of the Cave

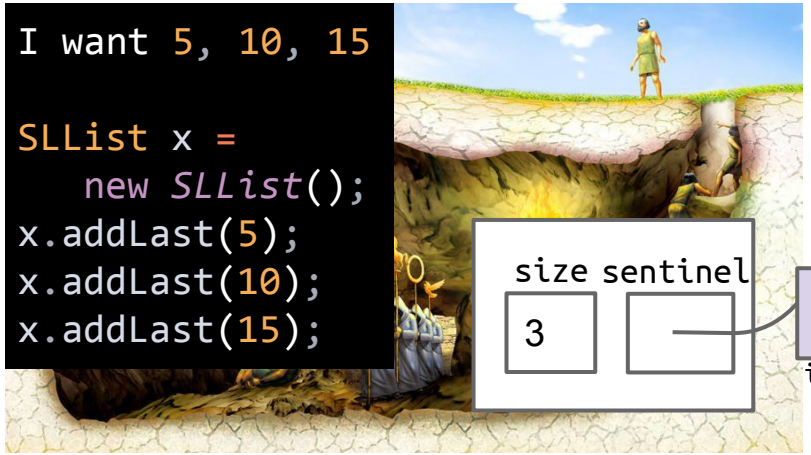
```
I want 5, 10, 15

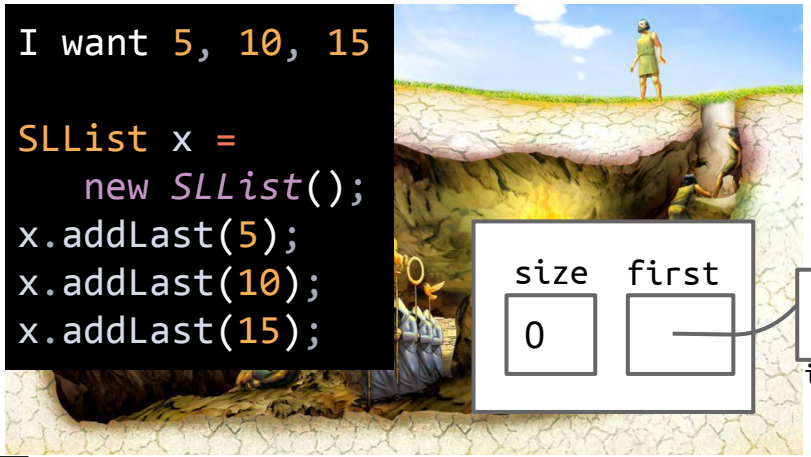
SLList x =
    new SLList();
x.addLast(5);
x.addLast(10);
x.addLast(15);
```



CC BY NC SA

Allegory of the Cave





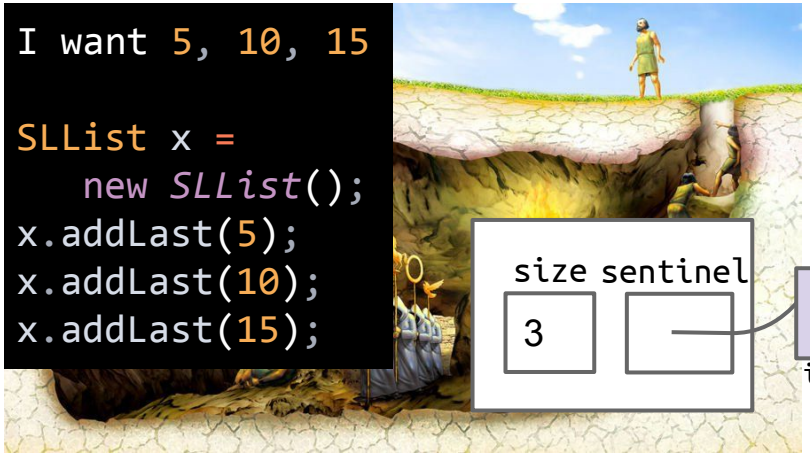
This would have been perfectly reasonable as well (but has the ugly special case that `first` is sometimes `null`).

CC BY NC SA

Allegory of the Cave

```
I want 5, 10, 15

SLList x =
    new SLList();
x.addLast(5);
x.addLast(10);
x.addLast(15);
```



This clean separation of desire and implementation is not possible with naked recursion (i.e. `IntList`).

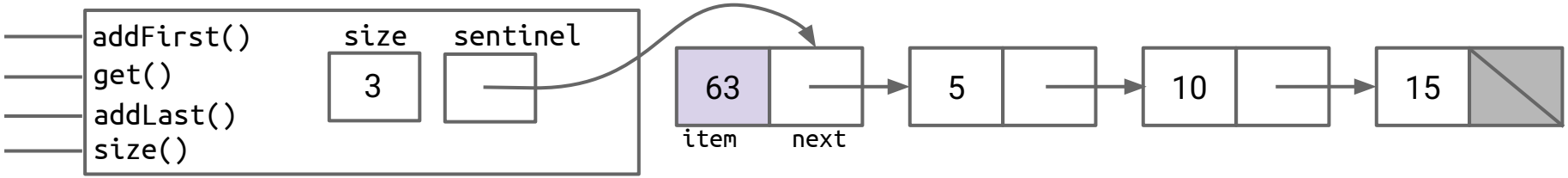
CC BY NC SA

Summary of Last Time (From IntList to SLList)

Last time: Non-Obvious Improvements	
#1	Rebranding: `IntList` → `IntNode`
#2	Bureaucracy: `SLList`
#3	Access Control: `public` → `private`
#4	Nested Class: Bringing `IntNode` into `SLList`
#5	Caching: Saving `size` as an `int`.
#6	Generalizing: Adding a `sentinel` node to allow representation of the empty list.

Last time: Remaining Problems

- Slow to get to the end of the list!
- Only stores ints
- Slow to access items in the middle of the list



CC BY NC SA

Slow to get to the End of the List

Inserting at the back of an `SLList` is much slower than the front.

```
public void addFirst(int x) {
   sentinel.next = new IntNode(x, sentinel.next);
}
```

```
public void addLast(int x) {
   size += 1;

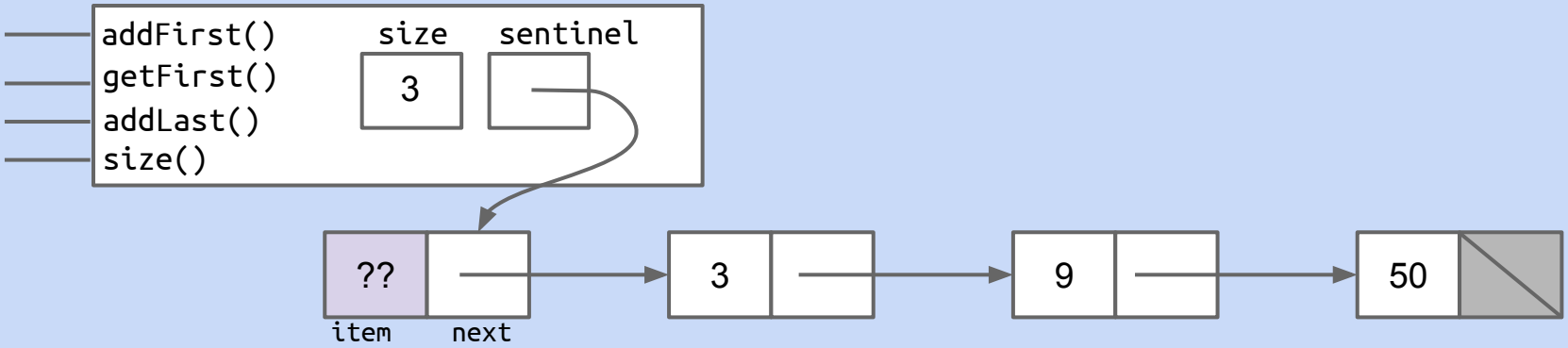
   IntNode p = sentinel;
   while (p.next != null) {
      p = p.next;
   }

   p.next = new IntNode(x, null);
}
```

cc BY NC SA

Improvement #7: (???) Goal: Fast `addLast`

How could we modify our list data structure so that `addLast` is also fast?

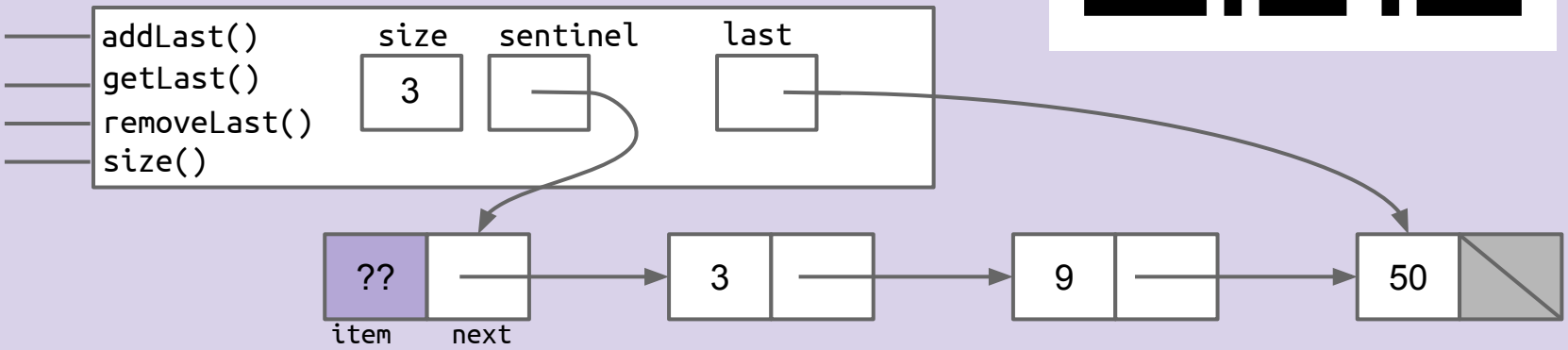


CC BY NC SA

Is .last enough? hugcode.com/robot

Suppose we want to support **addLast**, **getLast**, and **removeLast**, will having a `last` pointer make these methods fast, even on long lists?

A. Yes
B. No, addLast would be slow for long lists.
C. No, getLast would be slow for long lists.
D. No, removeLast would be slow for long lists.



CC BY NC SA

Why a Last Pointer Isn't Enough

Lecture 7, CS61B, Fall 2026

SLLists:

- Summary of SLLists So Far
- **Why a Last Pointer Isn't Enough**
- Doubly Linked Lists
- Generic Lists
- Linked Lists Are Bad at Get

ALists:

- Array Review
- removeLast and the Allegory of the Cave
- removeLast Implementation

CC BY NC SA

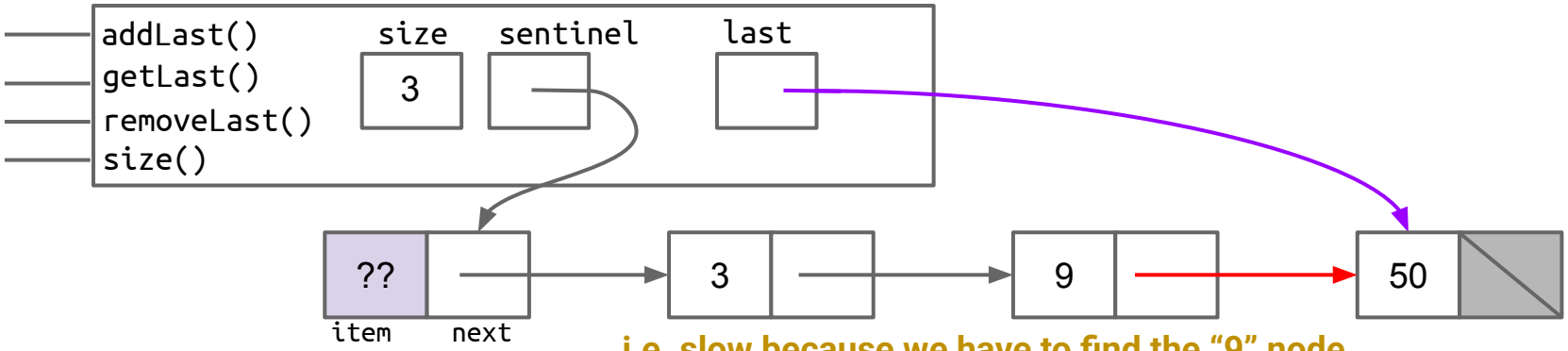
`.last` Is Not Enough

Suppose we want to support addLast, getLast, and removeLast, will having a last pointer make these methods fast, even on long lists?

- **No, removeLast would be slow for long lists.**

`removeLast` requires two actions:

- Setting 9's `next` variable to null.
- Setting `last` equal to 9's memory location.
  - Have to search through list to find the 9 node (second to last).



CC BY NC SA

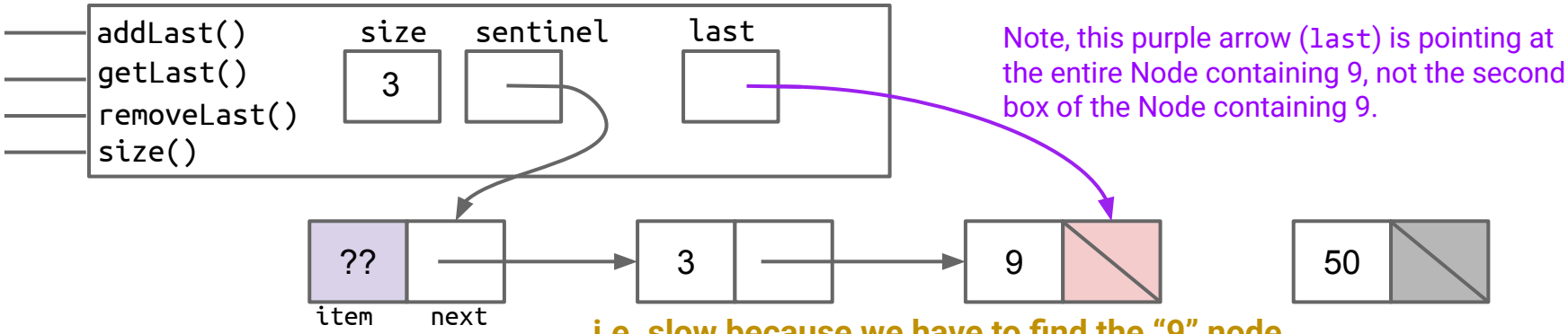
`.last` Is Not Enough

Suppose we want to support addLast, getLast, and removeLast, will having a last pointer make these methods fast, even on long lists?

- **No, removeLast would be slow for long lists.**

`removeLast` requires two actions:

- Setting 9's `next` variable to null.
- Setting `last` equal to 9's memory location.
  - Have to search through list to find the 9 node (second to last).

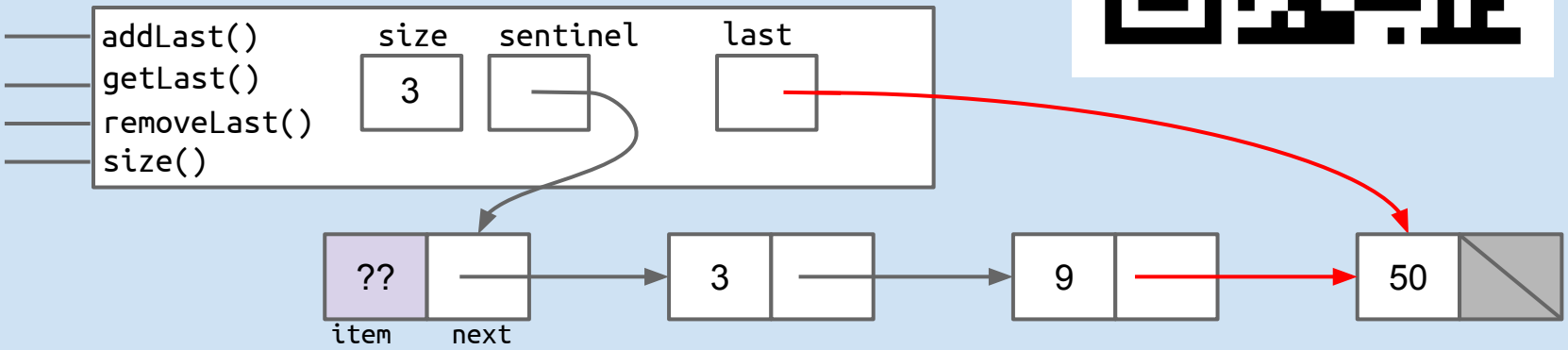


CC BY NC SA

Improvement #7: .last and ??? Goal: Fast operations on last.

We added `.last`, which will speed up `addLast` and `getLast`. What other changes might we make so that `removeLast` is also fast?

Poll:
hugcode.com/drum



CC BY NC SA

Doubly Linked Lists

Lecture 7, CS61B, Fall 2026

SLLists:

- Summary of SLLists So Far
- Why a Last Pointer Isn't Enough
- **Doubly Linked Lists**
- Generic Lists
- Linked Lists Are Bad at Get

ALists:

- Array Review
- Basic ArrayList Implementation
- removeLast and the Allegory of the Cave
- removeLast Implementation

CC BY NC SA

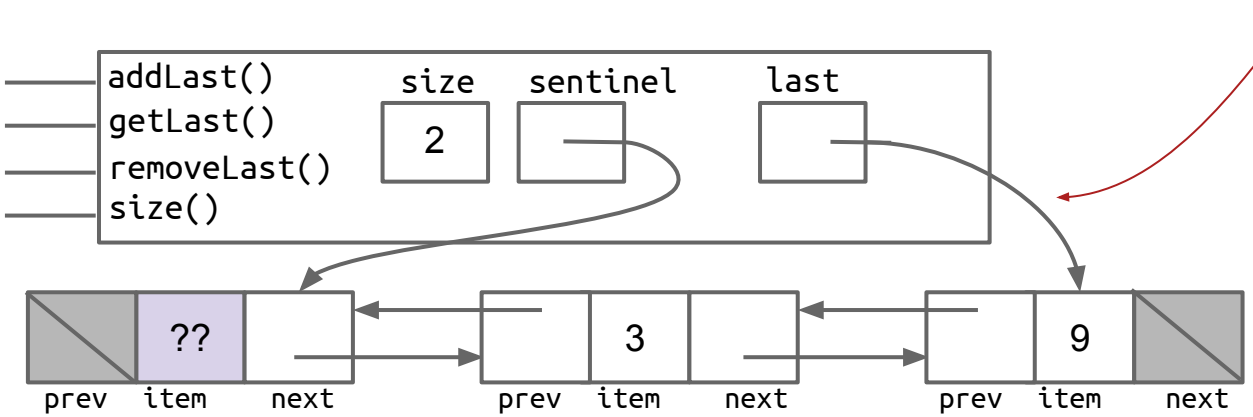
Improvement #7: .last and .prev

We added `.last`. What other changes might we make so that `removeLast` is also fast?

- Add backwards links from every node.
- This yields a "**doubly linked list**" or `DLList`, as opposed to our earlier "**singly linked list**" or `SLList`.

Note: Arrows point at entire nodes, not fields!

Example: last holds the address of the last node, not the item field of the sentinel node.

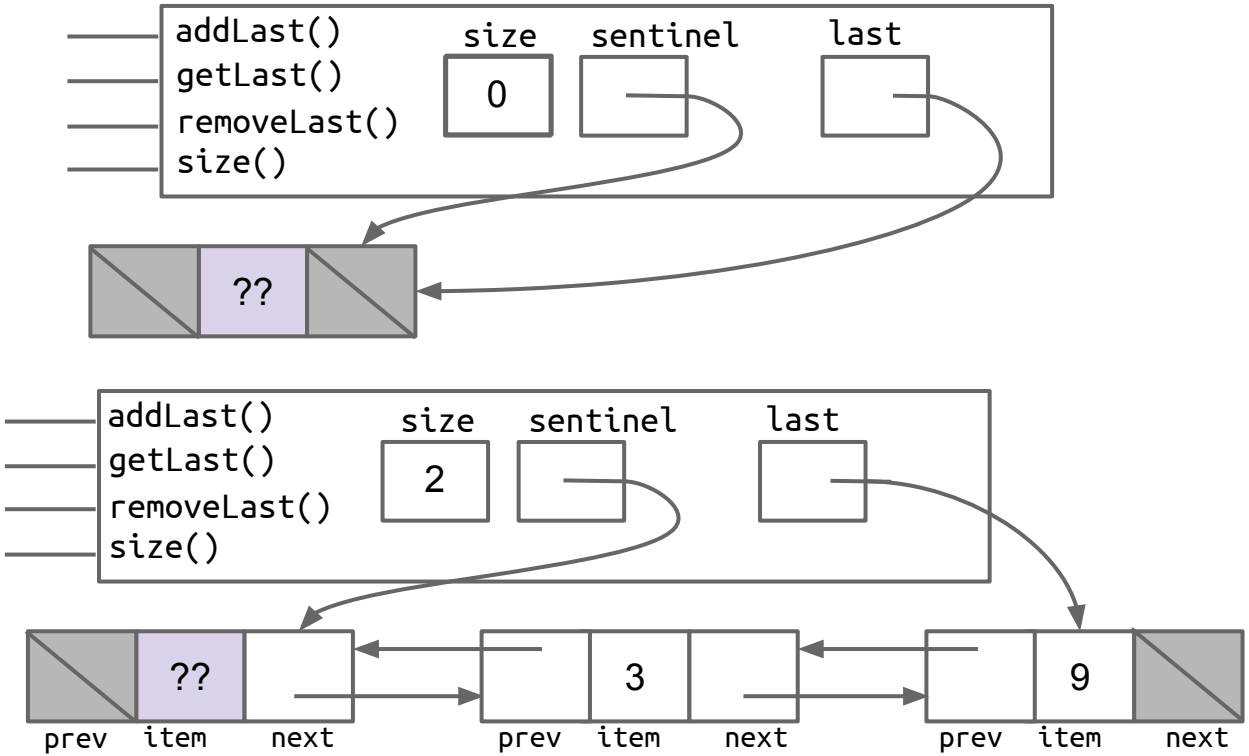


CC BY NC SA

Doubly Linked Lists (Naive)

Reverse pointers allow all operations (add, get, remove) to be fast.

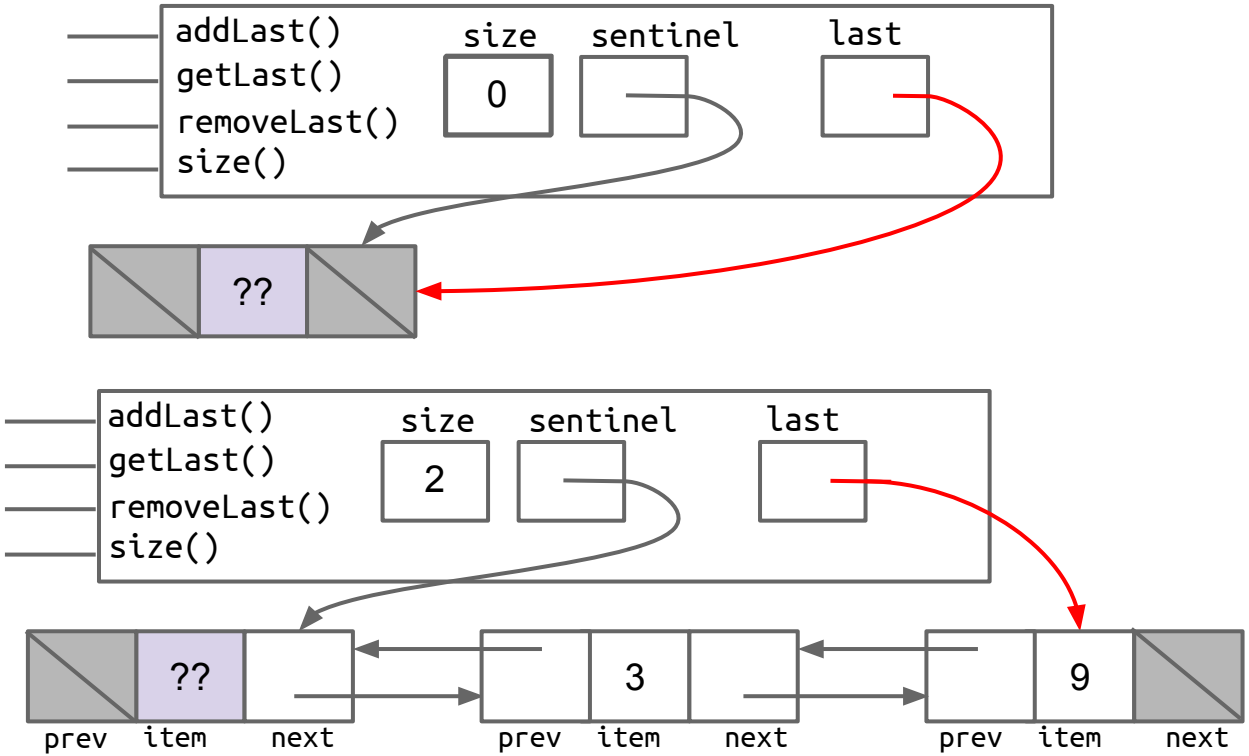
- We call such a list a “doubly linked list” or `DLList`.



CC BY NC SA

Doubly Linked Lists (Naive)

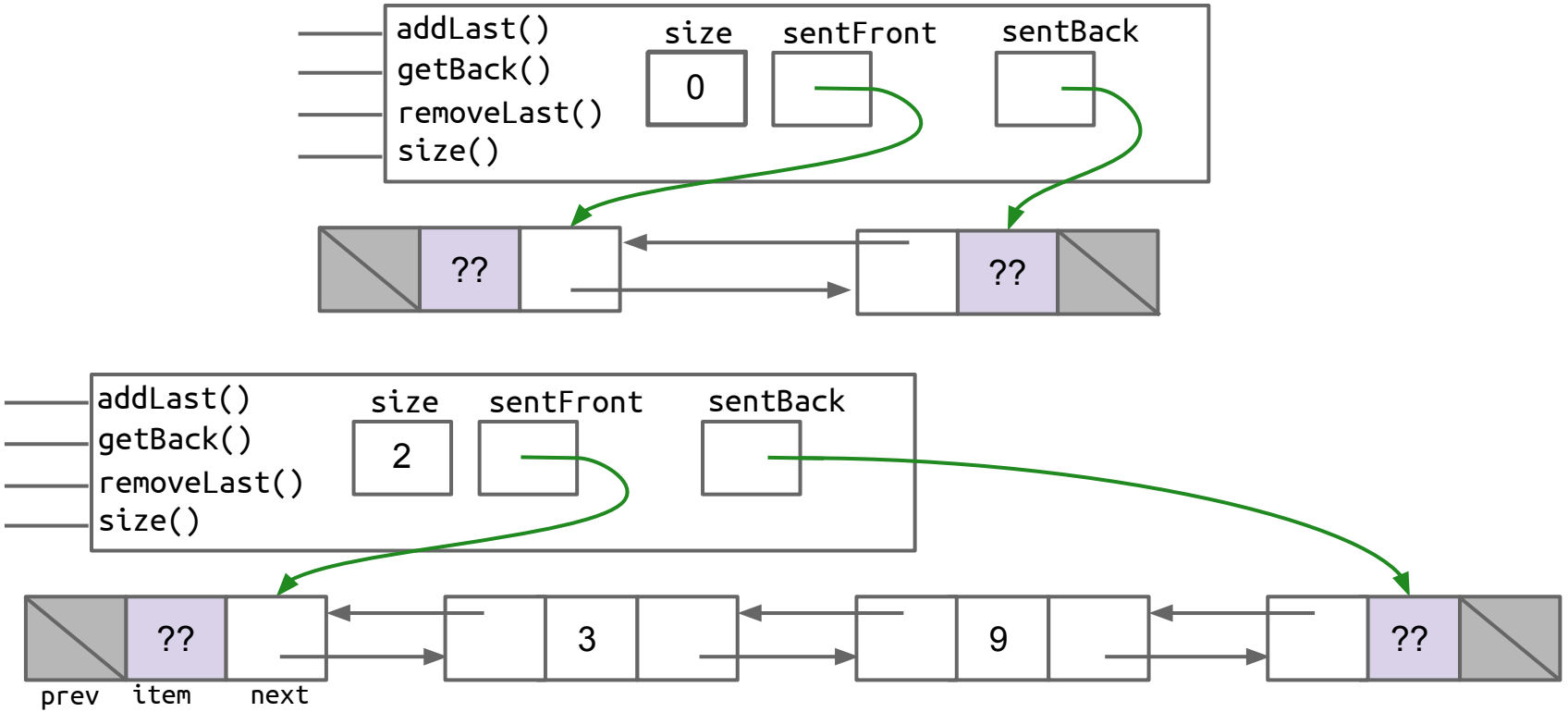
Non-obvious fact: This approach has an annoying special case: `last` sometimes points at the sentinel, and sometimes points at a 'real' node.



CC BY NC SA

Doubly Linked Lists (Double Sentinel)

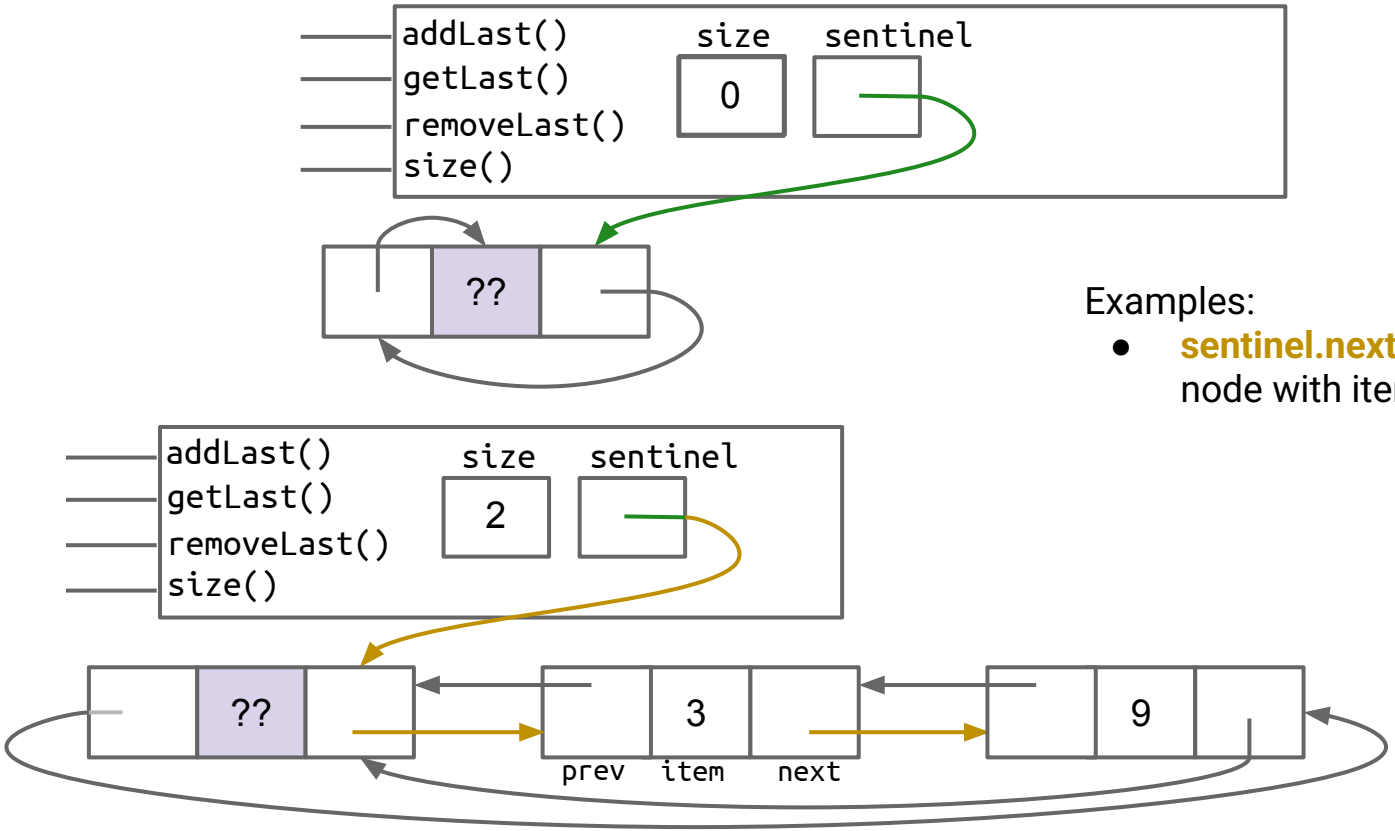
One solution: Have two sentinels.



CC BY NC SA

Doubly Linked Lists (Circular Sentinel)

Even better topology (IMO):



Examples:

- **sentinel.next.next** is the node with item=9.

This is the required approach for Project 1B.

CC BY NC SA

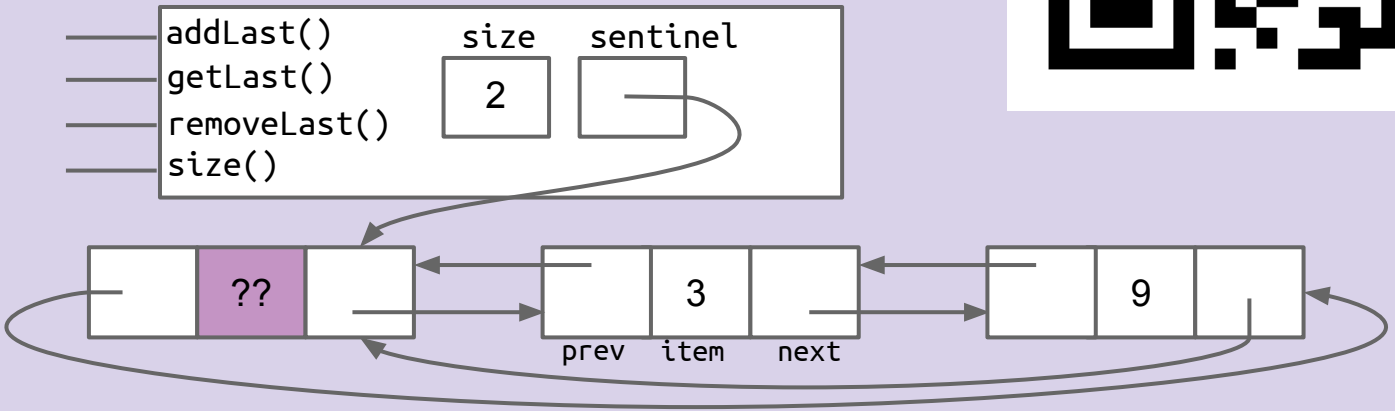
Doubly Linked Lists (Circular Sentinel) hugcode.com/tiger (also attendance)

How do we retrieve the first item in the list? We want the 3.

- sentinel.next.item

How do we retrieve the last item in the list?

- sentinel.prev.item



This is the required approach for Project 1B.

CC BY NC SA

Improvement #8: Fancier Sentinel Node(s)

While fast, adding `.last` and `.prev` introduces lots of special cases.

To avoid these, either:

- Add an additional `sentBack` sentinel at the end of the list.
- Make your linked list circular (required for Project 1B), with a single sentinel in the middle.

CC BY NC SA

DLList Summary

Last time: Non-Obvious Improvements	
#1	Rebranding: `IntList` → `IntNode`
#2	Bureaucracy: `SLList`
#3	Access Control: `public` → `private`
#4	Nested Class: Bringing `IntNode` into `SLList`
#5	Caching: Saving `size` as an `int`.
#6	Generalizing: Adding a `sentinel` node to allow representation of the empty list.

Last time: Remaining Problems

- Slow to get to the end of the list!
- Only stores ints
- Slow to access items in the middle of the list

CC BY NC SA

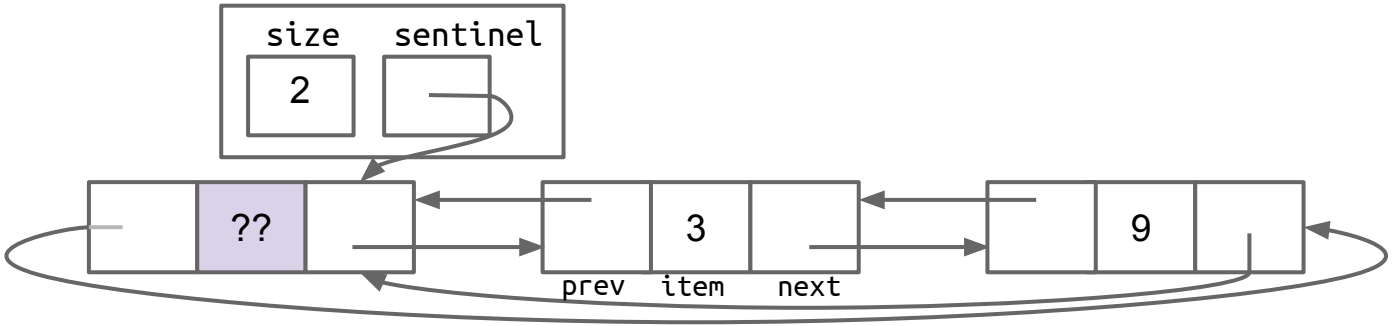
DLList Summary

Last time: Non-Obvious Improvements	
#1	Rebranding: `IntList` → `IntNode`
#2	Bureaucracy: `SLList`
#3	Access Control: `public` → `private`
#4	Nested Class: Bringing `IntNode` into `SLList`
#5	Caching: Saving `size` as an `int`.
#6	Generalizing: Adding a `sentinel` node to allow representation of the empty list.

Last time: Remaining Problems

- ~~Slow to get to the end of the list!~~
- Only stores ints
- Slow to access items in the middle of the list

DLList Improvements:	
#7	Looking back: `.last` and `.prev` allow fast `removeLast`
#8	Sentinel upgrade: Avoiding special cases with `sentBack` or circular list.



CC BY NC SA

DLList Summary

Methods	Non-Obvious Improvements	
`addFirst(int x)`	#1	Rebranding: `IntList` → `IntNode`
`get()`	#2	Bureaucracy: `SLList`
`addLast(int x)`	#3	Access Control: `public` → `private`
`size()`	#4	Nested Class: Bringing `IntNode` into `SLList`
	#5	Caching: Saving `size` as an `int`.
`removeLast()`	#6	Generalizing: Adding a `sentinel` node to allow representation of the empty list.
`getFirst()`	#7	Looking back: `.last` and `.prev` allow fast `removeLast`
	#8	Sentinel upgrade: Avoiding special cases with `sentBack` or circular list.

Still many steps before we have an industrial strength data structure. Will discuss over next two weeks.

CC BY NC SA

Generic Lists

Lecture 7, CS61B, Fall 2026

SLLists:

- Summary of SLLists So Far
- Why a Last Pointer Isn't Enough
- Doubly Linked Lists
- **Generic Lists**
- Linked Lists Are Bad at Get

ALists:

- Array Review
- Basic ArrayList Implementation
- removeLast and the Allegory of the Cave
- removeLast Implementation

CC BY NC SA

Integer Only Lists

One issue with our list classes: They only support integers.

```
public class SLList {
   private IntNode sentinel;
   private int size;


   public class IntNode {
      public int item;
      public IntNode next;
      ...
   }
   ...
}
```

```
SLList s1 = new SLList(5);
s1.addFirst(10);
```

Works fine!

```
SLList s2 = new SLList("hi");
s2.addFirst("apple");
```

```
SLListLauncher.java:6: error:
incompatible types: String cannot
be converted to int

SLList s2 = new SLList("hi");
```

CC BY NC SA

Coding Demo: Generic Lists

SLList.java

```
public class SLList {
   private IntNode sentinel;
   private int size;

   private class IntNode {
      public int item;
      public IntNode next;

      public IntNode(int i, IntNode n) {
         item = i;
         next = n;
      }
   }



}
```

In this demo, we'll modify our SLList to support lists of any data type, not just lists of integers.

CC BY NC SA

Coding Demo: Generic Lists

SLList.java

```
public class SLList<LochNess> {
   private IntNode sentinel;
   private int size;

   private class IntNode {
      public int item;
      public IntNode next;

      public IntNode(int i, IntNode n) {
         item = i;
         next = n;
      }
   }



}
```

A placeholder name, which will get replaced by the true data type each time a new SLList is created.

CC BY NC SA

Coding Demo: Generic Lists

SLList.java

```
public class SLList<LochNess> {
   private IntNode sentinel;
   private int size;

   private class IntNode {
      public LochNess item;
      public IntNode next;

      public IntNode(LochNess i, IntNode n) {
         item = i;
         next = n;
      }
   }



}
```

Items are no longer integers, but the LochNess placeholder data type.

CC BY NC SA

Coding Demo: Generic Lists

SLList.java

```
public class SLList<LochNess> {
   private StuffNode sentinel;
   private int size;

   private class StuffNode {
      public LochNess item;
      public StuffNode next;

      public StuffNode(LochNess i, StuffNode n) {
         item = i;
         next = n;
      }
   }



}
```

Renaming IntNode to StuffNode to be more descriptive.

CC BY NC SA

Coding Demo: Generic Lists

SLList.java

```
public class SLList<LochNess> {
   private StuffNode sentinel;
   private int size;

   public SLList(LochNess x) {
      sentinel = new StuffNode(null, null);
      sentinel.next = new StuffNode(x, null);
      size = 1;
   }

   public SLList() {
      sentinel = new StuffNode(null, null);
      size = 0;
   }


}
```

Replaced int x with LochNess x, the placeholder data type.

CC BY NC SA

Coding Demo: Generic Lists

SLList.java

```
public class SLList<LochNess> {
   private StuffNode sentinel;
   private int size;

   public void addFirst(LochNess x) {
      sentinel.next = new StuffNode(x, sentinel.next);
      size += 1;
   }

   public LochNess getFirst() {
      return sentinel.next.item;
   }




}
```

Replaced int x with LochNess x, the placeholder data type.

Return type is LochNess, not int.

CC BY NC SA

Coding Demo: Generic Lists

SLList.java

```
public class SLList<LochNess> {
   private StuffNode sentinel;
   private int size;

   public void addLast(LochNess x) {
      size += 1;
      StuffNode p = sentinel;

      /** Move p until it reaches the end of the list. */
      while (p.next != null) {
         p = p.next;
      }
      p.next = new StuffNode(x, null);
   }


}
```

Replaced int x with LochNess x, the placeholder data type.

CC BY NC SA

SLLists

Java allows us to defer type selection until declaration.

```
public class SLList<BleepBlorp> {
   private IntNode sentinel;
   private int size;

   public class IntNode {
      public BleepBlorp item;
      public IntNode next;
      ...
   }

   ...
}
```

```
SLList<Integer> s1 = new SLList<>(5);
s1.addFirst(10);

SLList<String> s2 = new SLList<>("hi");
s2.addFirst("apple");
```

CC BY NC SA

Generics

We won't get into the various complexities around generics in 61B. Here are the rules of thumb you'll need for our course:

- In the .java file **implementing** your data structure, specify your "generic type" **only once** at the very top of the file.
- In .java files that **use** your data structure, specify desired type **once**:
  - Write out desired type during **declaration**.
  - Use the empty diamond operator <> during **instantiation**.
- Make the nested class non-static (there's a way around this, ask at OH).
- When declaring or instantiating your data structure, use the reference type.
  - `int`: `Integer`
  - `double`: `Double`
  - `char`: `Character`
  - `boolean`: `Boolean`
  - `long`: `Long`
  - etc.
  - For more: see the textbook chapter for this lecture.

```
DLList<Double> s1 = new DLList<>(5.3);

double x = 9.3 + 15.2;
s1.addFirst(x);
```

CC BY NC SA

DLList Summary

Last time: Non-Obvious Improvements	
#1	Rebranding: `IntList` → `IntNode`
#2	Bureaucracy: `SLList`
#3	Access Control: `public` → `private`
#4	Nested Class: Bringing `IntNode` into `SLList`
#5	Caching: Saving `size` as an `int`.
#6	Generalizing: Adding a `sentinel` node to allow representation of the empty list.

Last time: Remaining Problems

- ~~Slow to get to the end of the list!~~
- Only stores ints
- Slow to access items in the middle of the list

DLList Improvements:	
#7	Looking back: `.last` and `.prev` allow fast `removeLast`
#8	Sentinel upgrade: Avoiding special cases with `sentBack` or circular list.

CC BY NC SA

DLList Summary

Last time: Non-Obvious Improvements	
#1	Rebranding: `IntList` → `IntNode`
#2	Bureaucracy: `SLList`
#3	Access Control: `public` → `private`
#4	Nested Class: Bringing `IntNode` into `SLList`
#5	Caching: Saving `size` as an `int`.
#6	Generalizing: Adding a `sentinel` node to allow representation of the empty list.

Last time: Remaining Problems

- ~~Slow to get to the end of the list!~~
- ~~Only stores ints~~
- Slow to access items in the middle of the list

DLList Improvements:	
#7	Looking back: `.last` and `.prev` allow fast `removeLast`
#8	Sentinel upgrade: Avoiding special cases with `sentBack` or circular list.
#9	Generics: DLList can store anything

CC BY NC SA

Linked Lists Are Bad at Get

Lecture 7, CS61B, Fall 2026

SLLists:

- Summary of SLLists So Far
- Why a Last Pointer Isn't Enough
- Doubly Linked Lists
- Generic Lists
- **Linked Lists Are Bad at Get**

ALists:

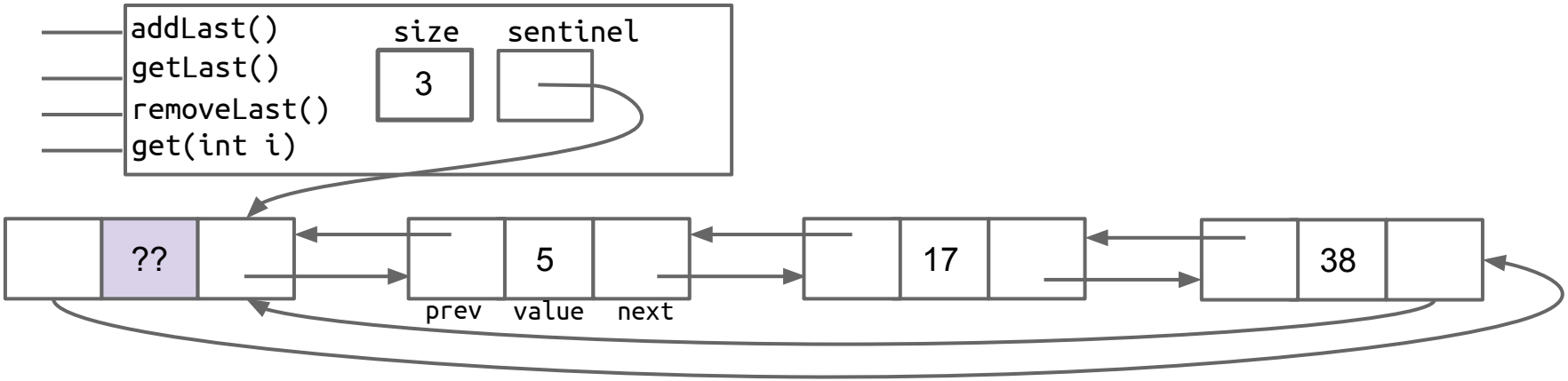
- Array Review
- Basic ArrayList Implementation
- removeLast and the Allegory of the Cave
- removeLast Implementation

CC BY NC SA

Doubly Linked Lists

Good news: We've designed a linked list that can handle the following operations efficiently.

- `addFirst`, `addLast`
- `getFirst`, `getLast`
- `removeFirst`, `removeLast`
- You will build this in Project 1B.



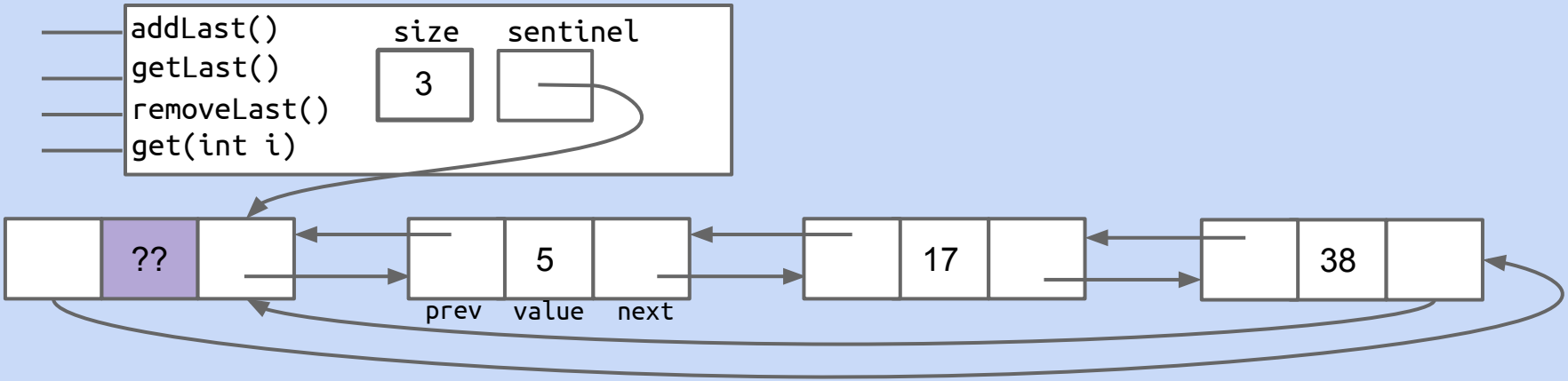
CC BY NC SA

Arbitrary Retrieval

Bad news: Can't efficiently implement `get(int i)`.

Suppose we added `get(int i)`, which returns the ith item from the list.

Why would `get` be slow for long lists compared to `getLast()`? For what inputs?



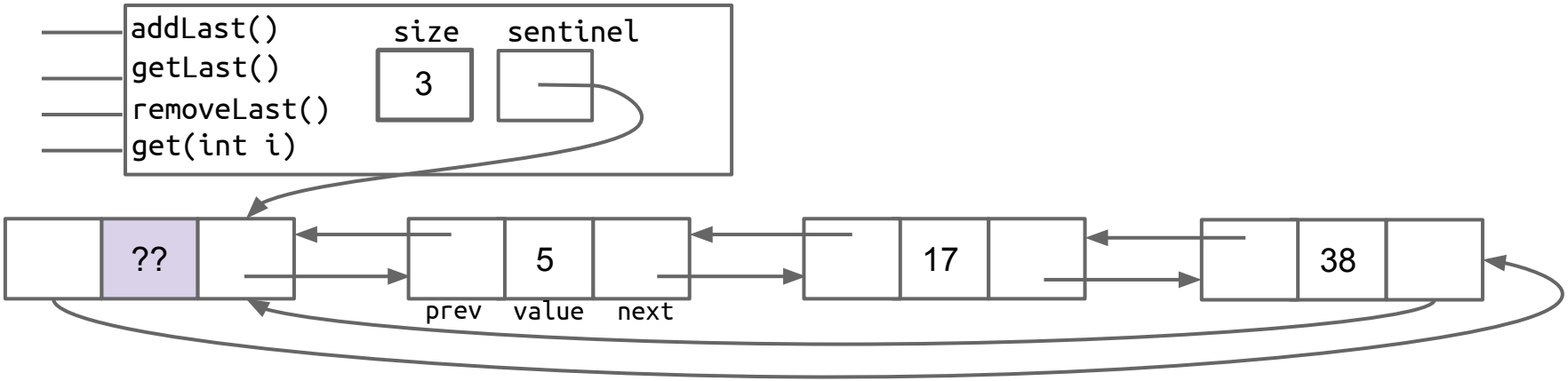
CC BY NC SA

Arbitrary Retrieval

Suppose we added `get(int i)`, which returns the ith item from the list.

Why would `get` be slow for long lists compared to `getLast()`? For what inputs?

- Have to scan to desired position. Slow for any `i` not near the sentinel node.
- How do we fix this?



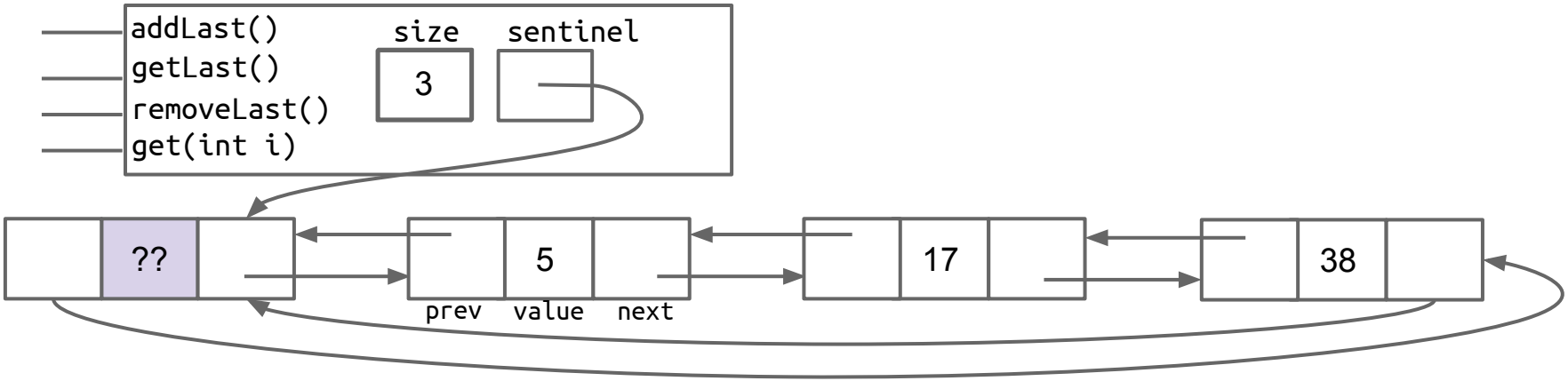
CC BY NC SA

Arbitrary Retrieval

Suppose we added `get(int i)`, which returns the ith item from the list.

Why would `get` be slow for long lists compared to `getLast()`? For what inputs?

- Have to scan to desired position. Slow for any `i` not near the sentinel node.
- Will discuss (much later) sophisticated changes that can speed things up.
- For today: We'll take a different tack: Using an array instead (no links!).



CC BY NC SA

Note

At this point we've covered everything you need to know to complete Project 1B.

- On Project 1B, you'll implement a doubly linked list.
- The project includes a bit of syntax we won't formally cover in lecture, specifically the idea of a "class implementing an interface". It isn't necessary to know what this means to complete the project.
- We will cover interfaces next Wednesday.
  - That is, what is this mysterious "Deque.java" that we've provided for Project 1A and 1B?
  - For Project 1A and 1B, just trust us!

CC BY NC SA

Array Overview

Lecture 7, CS61B, Fall 2026

SLLists:

- Summary of SLLists So Far
- Why a Last Pointer Isn't Enough
- Doubly Linked Lists
- Generic Lists
- Linked Lists Are Bad at Get

ALists:

- **Array Review**
- Basic ArrayList Implementation
- removeLast and the Allegory of the Cave
- removeLast Implementation

CC BY NC SA

Getting Memory Boxes

To store information, we need memory boxes, which we can get in Java by declaring variables or instantiating objects. Examples:

- `int x;` ← Gives us a memory box of 32 bits that stores ints.
- `Walrus w1;` ← Gives us a memory box of 64 bits that stores Walrus references.
- `Walrus w2 = new Walrus(30, 5.6);` ← Gives us a memory box of 64 bits that stores Walrus references, and also gives us 96 bits for storing the int size (32 bits) and double tuskSize (64 bits) of our Walrus.

Arrays are a special kind of object which consists of a **numbered** sequence of memory boxes.

- To get ith item of array `A`, use `A[i]`.
- Unlike **class** instances which have have **named** memory boxes.

CC BY NC SA

Arrays

Arrays consist of:

- A fixed integer **length** (cannot change!)
- A sequence of N memory boxes where **N=length**, such that:
  - All of the boxes hold the same type of value (and have same # of bits).
  - The boxes are numbered 0 through length-1.

Like instances of classes:

- You get one reference when its created.
- If you reassign all variables containing that reference, you can never get the array back.

Unlike classes, arrays do not have methods.

CC BY NC SA

Declaration and Instantiation of Arrays

Arrays are also Objects. As we've seen, objects are (usually) instantiated using the ***new*** keyword.

- `Planet p = new Planet(0, 0, 0, 0, 0, "blah.png");`
- `int[] x = new int[]{0, 1, 2, 95, 4};`

```
int[] a;
```

Declaration

- Declaration creates a 64 bit box intended only for storing a reference to an int array. **No object is instantiated.**



Instantiation (HW0 covers this syntax)

```
new int[]{0, 1, 2, 95, 4};
```



- Instantiates a new Object, in this case an int array.
- Object is anonymous!

Minor technical note: I'm abusing the term instantiate a little by applying it to arrays.

CC BY NC SA

Assignment of Arrays

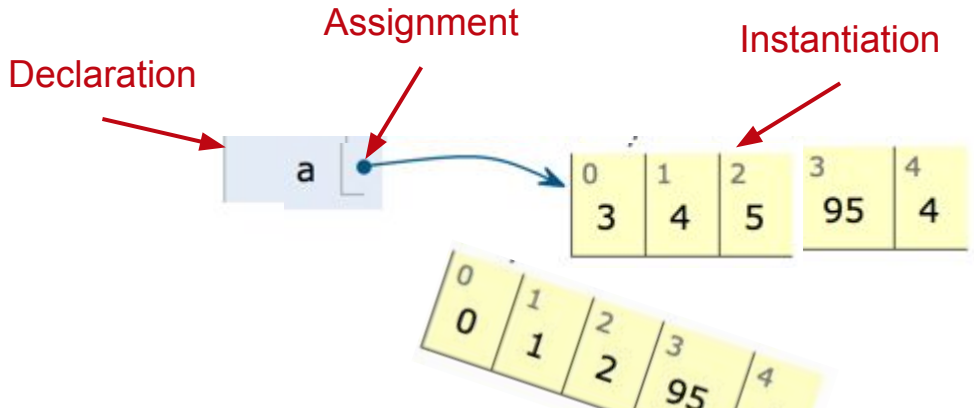
```
int[] a = new int[]{0, 1, 2, 95, 4};
```

Declaration, instantiation, and assignment.

- Creates a 64 bit box for storing an int array address. (declaration)
- Creates a new Object, in this case an int array. (instantiation)
- Puts the address of this new Object into the 64 bit box named a. (assignment)

Note: Instantiated objects can be lost!

- If we were to reassign a to something else, we'd never be able to get the original Object back!

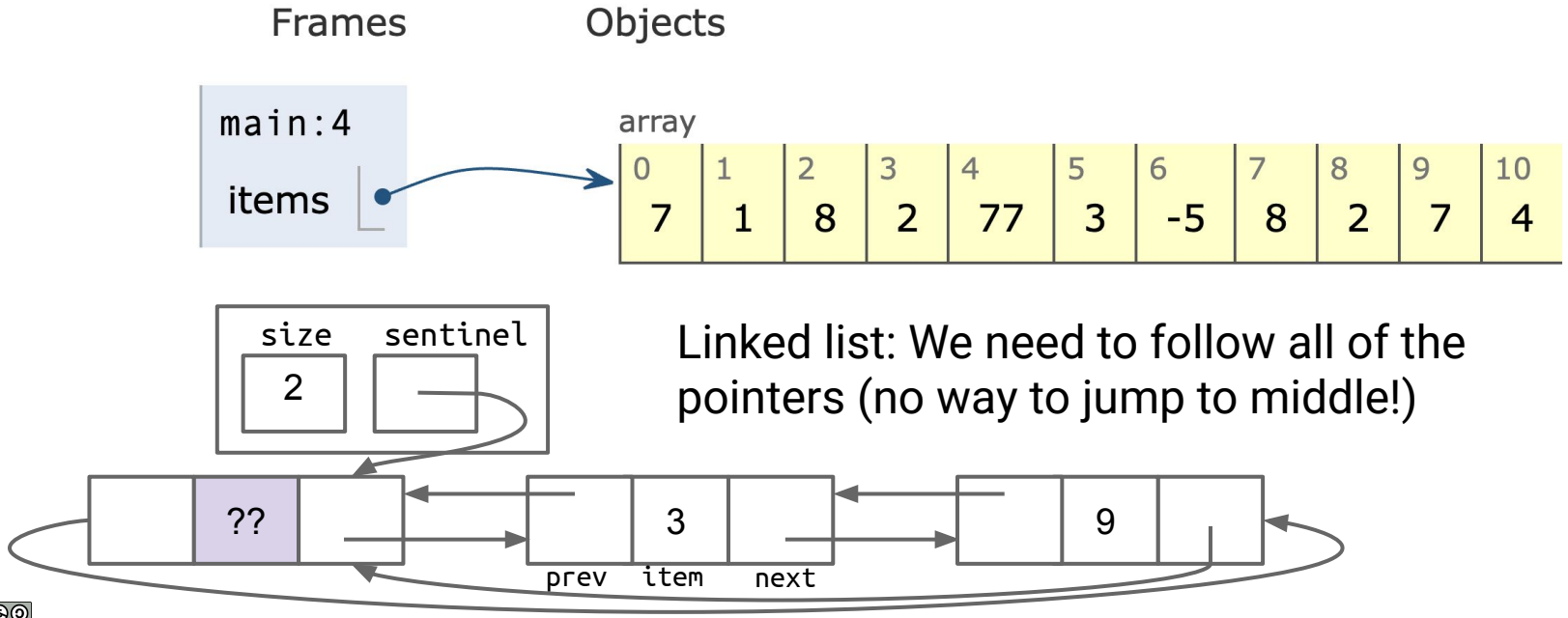


CC BY NC SA

Random Access in Arrays

Retrieval from any position of an array is very fast.

- Independent* of array size.
- 61C Preview: Ultra fast random access results from the fact that memory boxes are the same size (in bits).
- A[5000000] -- this is just fast



Linked list: We need to follow all of the pointers (no way to jump to middle!)

CC BY NC SA

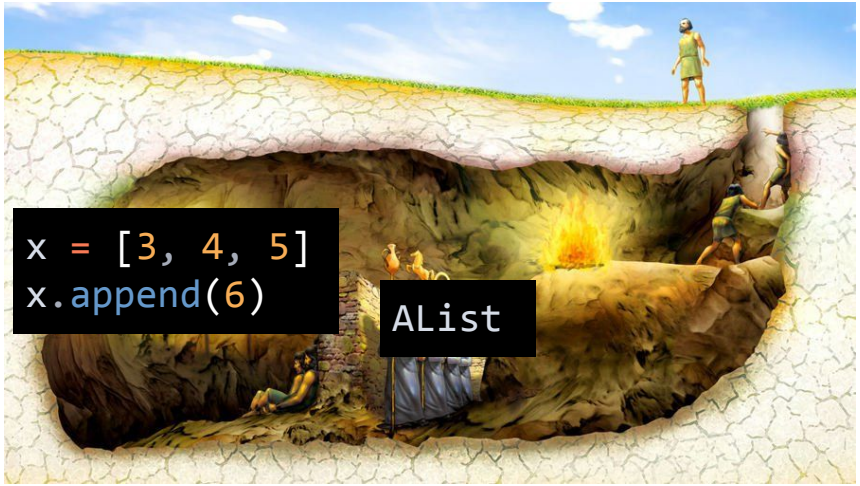
IntList, SLList, and DLList Summary

Non-Obvious Improvements	
#1	Rebranding: `IntList` → `IntNode`
#2	Bureaucracy: `SLList`
#3	Access Control: `public` → `private`
#4	Nested Class: Bringing `IntNode` into `SLList`
#5	Caching: Saving `size` as an `int`.
#6	Generalizing: Adding a `sentinel` node to allow representation of the empty list.
#7	Looking back: `.last` and `.prev` allow fast `removeLast`
#8	Sentinel upgrade: Avoiding special cases with `sentBack` or circular list.
#9	Generics: DLList can store anything

Last time: Remaining Problems

- ~~Slow to get to the end of the list!~~
- ~~Only stores ints~~
- **Slow to access items in the middle of the list**

Today (and Monday): AList
Implementing lists based on an array
(Python's approach)

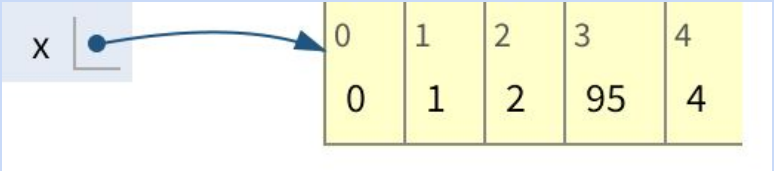


CC BY NC SA

Using Arrays to Build a List

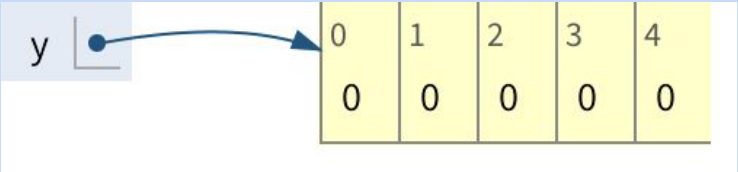
```
int[] x = new int[]{0, 1, 2, 95, 4};
```

- Instantiates an array with specific integers.




```
int[] y = new int[5];
```

- Instantiates an array of empty values.



Arrays have a fixed length. How can we use an Array to build a list?

CC BY NC SA

Basic ArrayList Implementation

Lecture 7, CS61B, Fall 2026

SLLists:

- Summary of SLLists So Far
- Why a Last Pointer Isn't Enough
- Doubly Linked Lists
- Generic Lists
- Linked Lists Are Bad at Get

ALists:

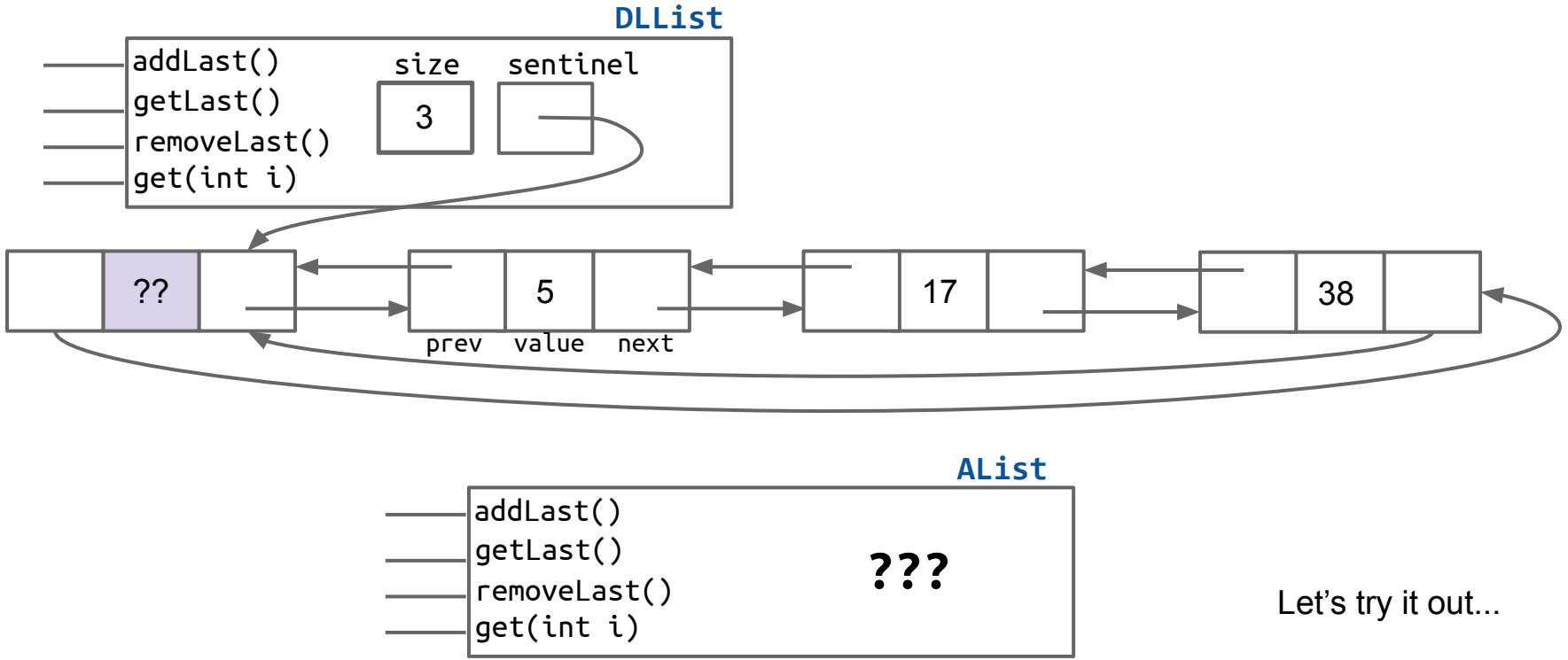
- Array Review
- **Basic ArrayList Implementation**
- removeLast and the Allegory of the Cave
- removeLast Implementation

CC BY NC SA

Our Goal: AList.java

Want to figure out how to build an array version of a list:

- In lecture we'll only do back operations. Project 2 will cover front operations.



Let's try it out...

CC BY NC SA

Coding Demo: Basic ArrayList Constructor

AList.java

```
public class AList {


   /** Creates an empty list. */
   public AList() {


   }


}
```

CC BY NC SA

Coding Demo: Basic ArrayList Constructor

AList.java

```
public class AList {

   private int size;

   /** Creates an empty list. */
   public AList() {


   }


}
```

CC BY NC SA

Coding Demo: Basic ArrayList Constructor

AList.java

```
public class AList {
   private int[] items;
   private int size;

   /** Creates an empty list. */
   public AList() {


   }


}
```

CC BY NC SA

Coding Demo: Basic ArrayList Constructor

AList.java

```
public class AList {
   private int[] items;
   private int size;

   /** Creates an empty list. */
   public AList() {
       items = new int[100];

   }


}
```

The choice of array size (999) was arbitrary. We'll fix this limitation later.

CC BY NC SA

Coding Demo: Basic ArrayList Constructor

AList.java

```
public class AList {
   private int[] items;
   private int size;

   /** Creates an empty list. */
   public AList() {
       items = new int[100];
       size = 0;
   }


}
```

CC BY NC SA

Coding Demo: Basic ArrayList addLast

AList.java

```
public class AList {
   private int[] items;
   private int size;

   /** Inserts x into the back of the list. */
   public void addLast(int x) {

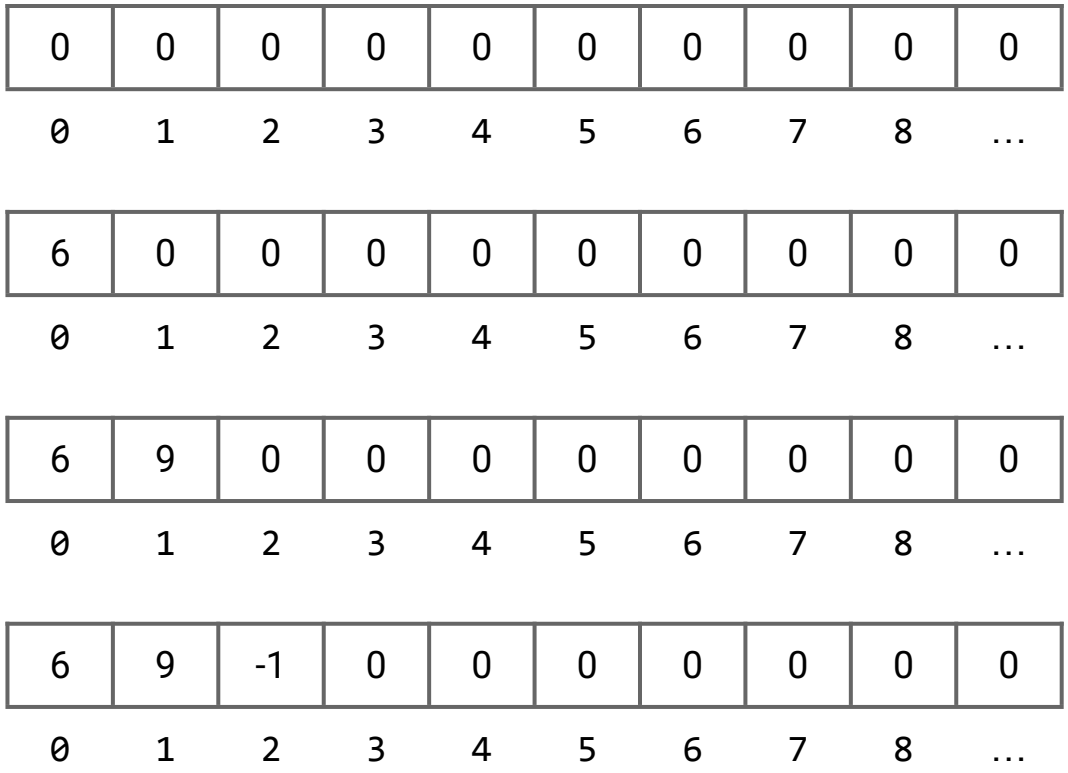

   }


}
```

Let's write a small example to help us think about addLast.

CC BY NC SA

Coding Demo: Basic ArrayList addLast



Call constructor to get empty array.

size=0

addLast(6)

size=1

addLast(9)

size=2

addLast(-1)

size=3

What patterns do we spot?

The next item we want to add will go into position size.

CC BY NC SA

Coding Demo: Basic ArrayList addLast

AList.java

```
/** Invariants:
    addLast: The next item we want to add, will go into position size


*/
public class AList {
   private int[] items;
   private int size;

   /** Inserts x into the back of the list. */
   public void addLast(int x) {


   }


}
```

CC BY NC SA

Coding Demo: Basic ArrayList addLast

AList.java

```
/** Invariants:
    addLast: The next item we want to add, will go into position size


*/
public class AList {
   private int[] items;
   private int size;

   /** Inserts x into the back of the list. */
   public void addLast(int x) {
       items[size] = x;

   }


}
```

CC BY NC SA

Coding Demo: Basic ArrayList addLast

AList.java

```
/** Invariants:
    addLast: The next item we want to add, will go into position size


*/
public class AList {
   private int[] items;
   private int size;

   /** Inserts x into the back of the list. */
   public void addLast(int x) {
       items[size] = x;
       size += 1;
   }


}
```

CC BY NC SA

Coding Demo: Basic ArrayList getLast

AList.java

```
/** Invariants:
    addLast: The next item we want to add, will go into position size


*/
public class AList {
   private int[] items;
   private int size;

   /** Returns the item from the back of the list. */
   public int getLast() {

   }



}
```

CC BY NC SA

Coding Demo: Basic ArrayList getLast

AList.java

```
/** Invariants:
    addLast: The next item we want to add, will go into position size
    getLast: The item we want to return is in position size - 1.

*/
public class AList {
   private int[] items;
   private int size;

   /** Returns the item from the back of the list. */
   public int getLast() {

   }



}
```

CC BY NC SA

Coding Demo: Basic ArrayList getLast

AList.java

```
/** Invariants:
    addLast: The next item we want to add, will go into position size
    getLast: The item we want to return is in position size - 1.

*/
public class AList {
   private int[] items;
   private int size;

   /** Returns the item from the back of the list. */
   public int getLast() {
       return items[size - 1];
   }



}
```

CC BY NC SA

Coding Demo: Basic ArrayList get

AList.java

```
/** Invariants:
    addLast: The next item we want to add, will go into position size
    getLast: The item we want to return is in position size - 1.

*/
public class AList {
   private int[] items;
   private int size;

   /** Gets the ith item in the list (0 is the front). */
   public int get(int i) {

   }



}
```

CC BY NC SA

Coding Demo: Basic ArrayList get

AList.java

```
/** Invariants:
    addLast: The next item we want to add, will go into position size
    getLast: The item we want to return is in position size - 1.

*/
public class AList {
   private int[] items;
   private int size;

   /** Gets the ith item in the list (0 is the front). */
   public int get(int i) {
       return items[i];
   }



}
```

CC BY NC SA

Coding Demo: Basic ArrayList size

AList.java

```
/** Invariants:
    addLast: The next item we want to add, will go into position size
    getLast: The item we want to return is in position size - 1.

*/
public class AList {
   private int[] items;
   private int size;

   /** Returns the number of items in the list. */
   public int size() {

   }



}
```

CC BY NC SA

Coding Demo: Basic ArrayList size

AList.java

```
/** Invariants:
    addLast: The next item we want to add, will go into position size
    getLast: The item we want to return is in position size - 1.
    size: The number of items in the list should be size.
*/
public class AList {
   private int[] items;
   private int size;

   /** Returns the number of items in the list. */
   public int size() {

   }



}
```

CC BY NC SA

Coding Demo: Basic ArrayList size

AList.java

```
/** Invariants:
    addLast: The next item we want to add, will go into position size
    getLast: The item we want to return is in position size - 1.
    size: The number of items in the list should be size.
*/
public class AList {
   private int[] items;
   private int size;

   /** Returns the number of items in the list. */
   public int size() {
       return size;
   }



}
```

CC BY NC SA

Invariants

An invariant is a condition that is guaranteed to be true during code execution (assuming no bugs).

An `AList` with a sentinel node has at least the following invariants:

- The position of the next item to be inserted is always `size`.
- `size` is always the number of items in the `AList`.
- The last item in the list is always in position `size - 1`.

Invariants make it easier to reason about code:

- Can assume they are true to simplify code
- Must ensure that methods preserve invariants.

```
public class AList {
   private int[] items;
   private int size;

   public AList() {
       items = new int[100];  size = 0;
   }

   public void addLast(int x) {
       items[size] = x;
       size += 1;
   }

   public int get(int i) {
       return items[i];
   }

   public int size() {
       return size;
   }
}
```

CC BY NC SA

removeLast and the Allegory of the Cave

Lecture 7, CS61B, Fall 2026

SLLists:

- Summary of SLLists So Far
- Why a Last Pointer Isn't Enough
- Doubly Linked Lists
- Generic Lists
- Linked Lists Are Bad at Get

ALists:

- Array Review
- Basic ArrayList Implementation
- **removeLast and the Allegory of the Cave**
- removeLast Implementation

CC BY NC SA

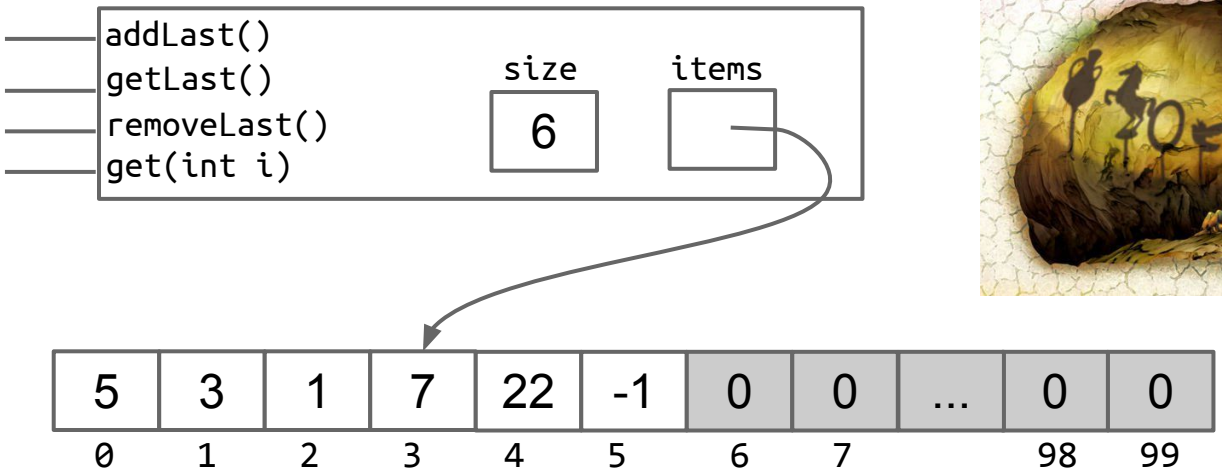
The Abstract vs. the Concrete

When we `removeLast()`, which memory boxes need to change? To what?-

User's mental model: `[5, 3, 1, 7, 22, -1]` → `[5, 3, 1, 7, 22]`



Actual truth:



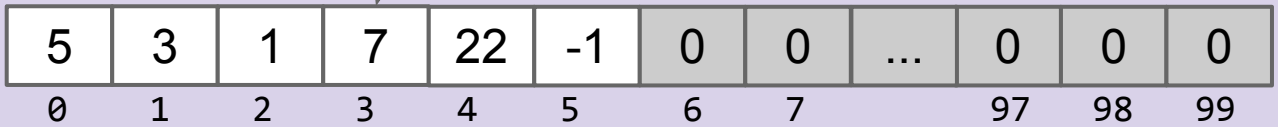
CC BY NC SA

Deletion: hugcode.com/bread

When we `removeLast()`, which memory boxes need to change? To what?



a) `size`
b) `size` and `items`
c) `size` and `items[i]` for some `i`
d) `size`, `items`, and `items[i]` for some `i`
e) `size`, `items`, and `items[i]` for many different `i`



- The position of the next item to be inserted is always `size`.
- size is always the number of items in the AList.
- The last item in the list is always in position `size - 1`.

AList invariants.

CC BY NC SA

removeLast Implementation

Lecture 7, CS61B, Fall 2026

SLLists:

- Summary of SLLists So Far
- Why a Last Pointer Isn't Enough
- Doubly Linked Lists
- Generic Lists
- Linked Lists Are Bad at Get

ALists:

- Array Review
- Basic ArrayList Implementation
- removeLast and the Allegory of the Cave
- **removeLast Implementation**

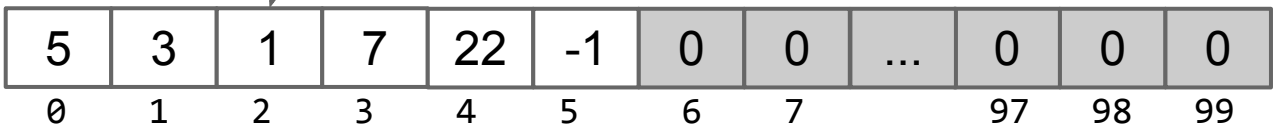
CC BY NC SA

Deletion Debrief

When we `removeLast()`, which memory boxes need to change? To what?



a) **`size`**
b) `size` and `items`
c) `size` and `items[i]` for some `i`
d) `size`, `items`, and `items[i]` for some `i`
e) `size`, `items`, and `items[i]` for many different `i`



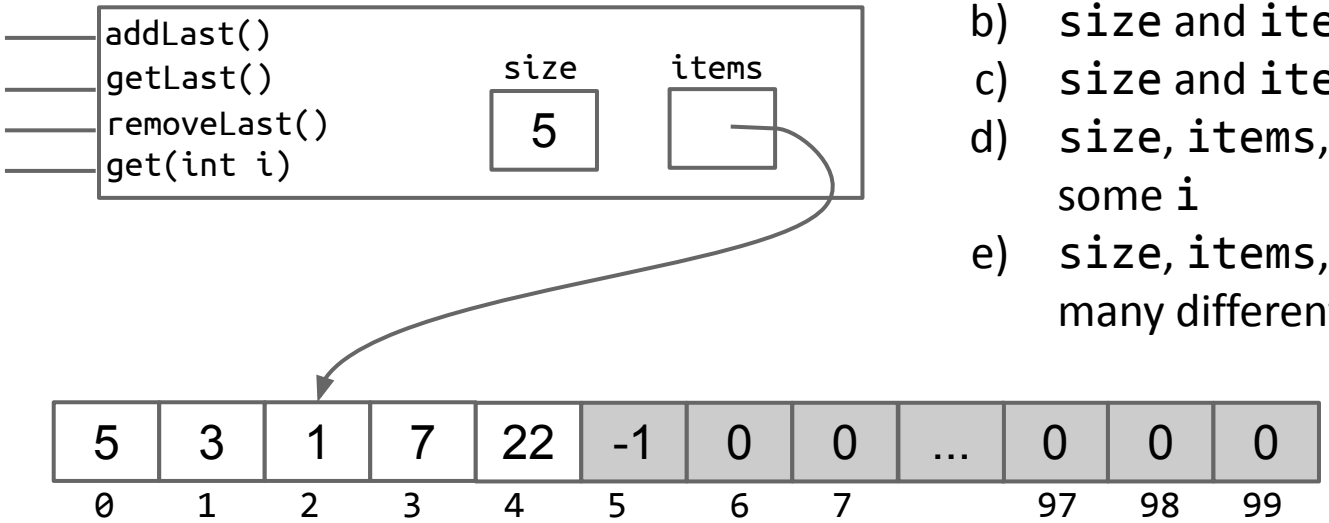
- The position of the next item to be inserted is always `size`.
- size is always the number of items in the AList.
- The last item in the list is always in position `size - 1`.

AList invariants.

CC BY NC SA

Deletion Debrief

When we `removeLast()`, which memory boxes need to change? To what?



a) **`size`**
b) `size` and `items`
c) `size` and `items[i]` for some `i`
d) `size`, `items`, and `items[i]` for some `i`
e) `size`, `items`, and `items[i]` for many different `i`

- The position of the next item to be inserted is always `size`.
- size is always the number of items in the AList.
- The last item in the list is always in position `size - 1`.

} AList invariants.

CC BY NC SA

Coding Demo: Basic ArrayList removeLast

AList.java

```
/** Invariants:
    addLast: The next item we want to add, will go into position size
    getLast: The item we want to return is in position size - 1.
    size: The number of items in the list should be size.
*/
public class AList {
   private int[] items;
   private int size;

   /** Deletes item from back of list and returns deleted item. */
   public int removeLast() {



   }

}
```

CC BY NC SA

Coding Demo: Basic ArrayList removeLast

AList.java

```
/** Invariants:
    addLast: The next item we want to add, will go into position size
    getLast: The item we want to return is in position size - 1.
    size: The number of items in the list should be size.
*/
public class AList {
   private int[] items;
   private int size;

   /** Deletes item from back of list and returns deleted item. */
   public int removeLast() {
       int x = getLast();



   }

}
```

CC BY NC SA

Coding Demo: Basic ArrayList removeLast

AList.java

```
/** Invariants:
    addLast: The next item we want to add, will go into position size
    getLast: The item we want to return is in position size - 1.
    size: The number of items in the list should be size.
*/
public class AList {
   private int[] items;
   private int size;

   /** Deletes item from back of list and returns deleted item. */
   public int removeLast() {
       int x = getLast();

       return x;
   }

}
```

CC BY NC SA

Coding Demo: Basic ArrayList removeLast

AList.java

```
/** Invariants:
    addLast: The next item we want to add, will go into position size
    getLast: The item we want to return is in position size - 1.
    size: The number of items in the list should be size.
*/
public class AList {
   private int[] items;
   private int size;

   /** Deletes item from back of list and returns deleted item. */
   public int removeLast() {
       int x = getLast();
       size = size - 1;
       return x;
   }

}
```

CC BY NC SA

Naive AList Code

From SLList lecture: “things that must be true”.

```
public class AList {
   private int[] items;
   private int size;

   public AList() {
       items = new int[100];  size = 0;
   }

   public void addLast(int x) {
       items[size] = x;
       size += 1;
   }

   public int getLast() {
       return items[size - 1];
   }

   public int get(int i) {
       return items[i];
   }

   public int size() {
       return size;
   }
}
```

AList **Invariants**:

- The position of the next item to be inserted is always `size`.
- `size` is always the number of items in the `AList`.
- The last item in the list is always in position `size - 1`.

```
public int removeLast() {
   int x = items[size - 1];
   items[size - 1] = 0;
   size -= 1;
   return x;
}
```

Setting deleted item to zero is not necessary to preserve invariants, and thus not necessary for correctness.

CC BY NC SA

Naive AList Code

From SLList lecture: “things that must be true”.

```
public class AList {
   private int[] items;
   private int size;

   public AList() {
       items = new int[100];  size = 0;
   }

   public void addLast(int x) {
       items[size] = x;
       size += 1;
   }

   public int getLast() {
       return items[size - 1];
   }

   public int get(int i) {
       return items[i];
   }

   public int size() {
       return size;
   }
}
```

AList **Invariants**:

- The position of the next item to be inserted is always `size`.
- `size` is always the number of items in the `AList`.
- The last item in the list is always in position `size - 1`.

We could also add error checking code, e.g.

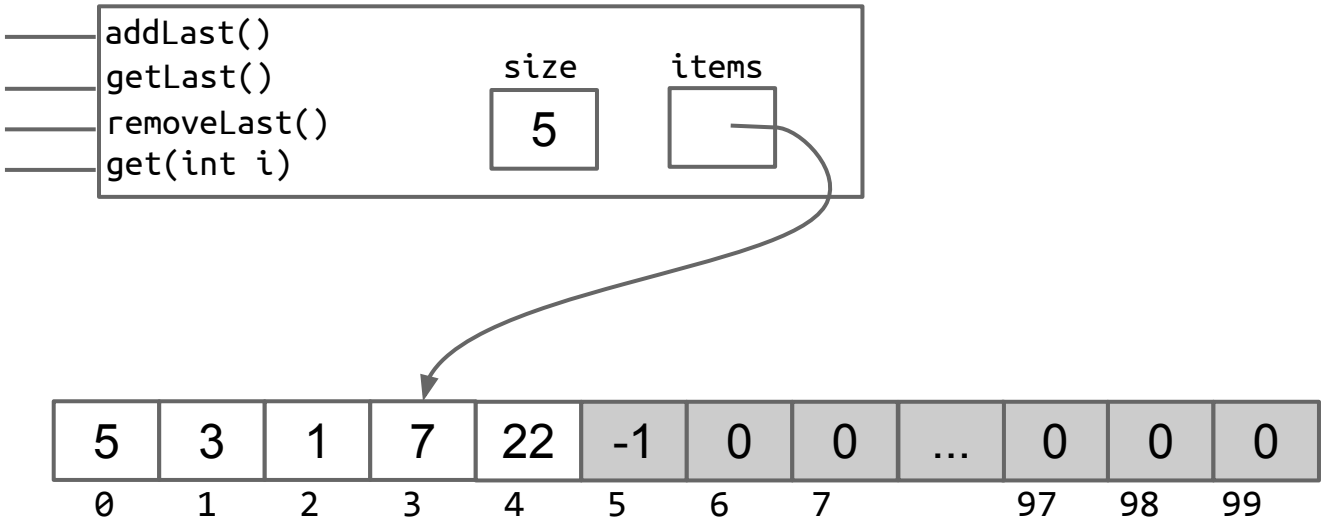
```
public int get(int i) {
   if (i >= items.length) {
       throw new IllegalArgumentException();
   }
   return items[i];
}
```

CC BY NC SA

Note

What about get?

- Some students suggest we should set the value to zero so that we can't get(5).
- There's no specified behavior for what to do when get is out of bounds.
  - IMO an exception is best as shown earlier.



CC BY NC SA

Naive AList Code

```
public class AList {
   private int[] items;
   private int size;

   public AList() {
       items = new int[100];  size = 0;
   }

   public void addLast(int x) {
       items[size] = x;
       size += 1;
   }

   public int get(int i) {
       return items[i];
   }

   public int size() {
       return size;
   }
}
```

What's missing from our implementation?

```
public int removeLast() {
   int x = items[size - 1];
   items[size - 1] = 0;
   size -= 1;
   return x;
}
```

cc BY NC SA

Naive AList Code

```
public class AList {
   private int[] items;
   private int size;

   public AList() {
       items = new int[100];  size = 0;
   }

   public void addLast(int x) {
       items[size] = x;
       size += 1;
   }

   public int get(int i) {
       return items[i];
   }

   public int size() {
       return size;
   }
}
```

What's missing from our implementation?

```
public int removeLast() {
   int x = items[size - 1];
   items[size - 1] = 0;
   size -= 1;
   return x;
}
```

CC BY NC SA

AList vs. DLList

IntList, SLList, and DLList Improvements	
#1	Rebranding: `IntList` → `IntNode`
#2	Bureaucracy: `SLList`
#3	Access Control: `public` → `private`
#4	Nested Class: Bringing `IntNode` into `SLList`
#5	Caching: Saving `size` as an `int`.
#6	Generalizing: Adding a `sentinel` node to allow representation of the empty list.
#7	Looking back: `.last` and `.prev` allow fast `removeLast`
#8	Sentinel upgrade: Avoiding special cases with `sentBack` or circular list.
#9	Generics: DLList can store anything

Last time: DLList Remaining Problems

- ~~Slow to get to the end of the list!~~
- ~~Only stores ints~~
- **Slow to access items in the middle of the list**

AList Improvements	
#1	Fast to access items anywhere in the list

CC BY NC SA

Naive AList Code

```
public class AList {
   private int[] items;
   private int size;

   public AList() {
       items = new int[100];  size = 0;
   }

   public void addLast(int x) {
       items[size] = x;
       size += 1;
   }

   public int get(int i) {
       return items[i];
   }

   public int size() {
       return size;
   }
}
```

In Lecture 8 (Monday), we'll cover:

- "Expanding" our array to support more than 100 items when necessary.
- Making our AList generic.

```
public int removeLast() {
   int x = items[size - 1];
   items[size - 1] = 0;
   size -= 1;
   return x;
}
```

CC BY NC SA

Reminders

Project 1A due today (9/11). Project 1B due Wednesday 9/16.

CC BY NC SA

Bonus Topic: Arrays vs. Classes

Lecture 7, CS61B, Fall 2026

CC BY NC SA

Arrays vs. Classes

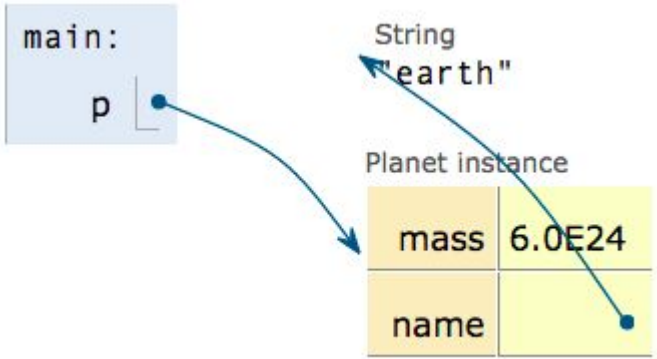
Arrays and Classes can both be used to organize a bunch of memory boxes.

- Array boxes are accessed using [ ] notation.
- Class boxes are accessed using dot notation.
- Array boxes must all be of the same type.
- Class boxes may be of different types.
- Both have a fixed number of boxes.

```
public class Planet {
   public double mass;
   public String name;
   ...
}
```

```
int[] x = new int[]{100, 101, 102, 103};
Planet p = new Planet(6e24, "earth");
```





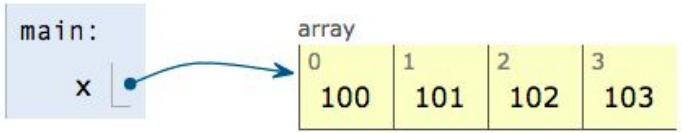
CC BY NC SA

Arrays vs. Classes

Array indices can be computed at runtime.

```
int[] x = new int[]{100, 101, 102, 103};
int indexOfInterest = askUser();
int k = x[indexOfInterest];
System.out.println(k);
```

```
jug ~/Dropbox/61b/lec/lists3
$ javac ArrayDemo.java
$ java ArrayDemo
What index do you want? 2
102
```



CC BY NC SA

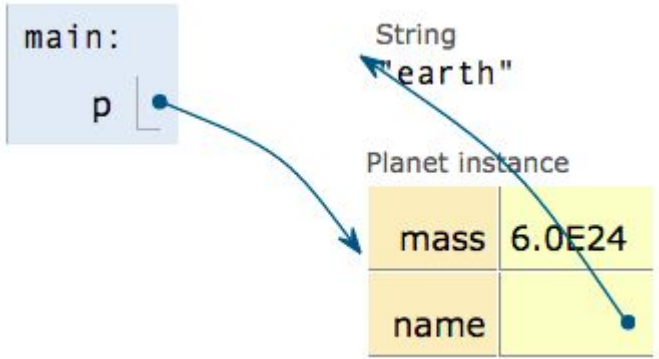
Arrays vs. Classes

Class member variable names CANNOT be computed and used at runtime.

```
String fieldOfInterest = "mass";
Planet earth = new Planet(6e24, "earth");
double mass = earth[fieldOfInterest];
System.out.println(mass);
```

```
jug ~/Dropbox/61b/lec/lists3
$ javac ClassDemo.java
ClassDemo.java:5: error: array required,
   but Planet found.

   double mass = earth[fieldOfInterest];
                       ^
```



… if you reallllly want to do this, you can:
https://docs.oracle.com/javase/tutorial/reflect/member/fieldValues.html

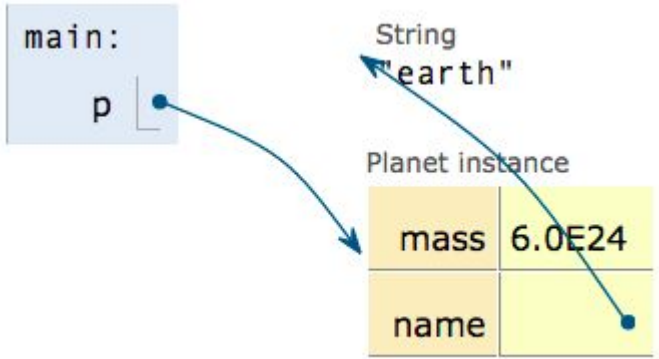
Arrays vs. Classes

Class member variable names CANNOT be computed and used at runtime.

- Dot notation doesn't work either.

```
String fieldOfInterest = "mass";
Planet earth = new Planet(6e24, "earth");
double mass = earth.fieldOfInterest;
System.out.println(mass);
```

```
jug ~/Dropbox/61b/lec/lists3
$ javac ClassDemo.java
ClassDemo.java:5: error: cannot find Symbol
   double mass = earth.fieldOfInterest;
                       ^
   symbol:   variable fieldOfInterest
   location: variable earth of type Planet
```



… if you reallllly want to do this, you can:
https://docs.oracle.com/javase/tutorial/reflect/member/fieldValues.html

CC BY NC SA

Another view

The only (easy) way to access a member of a class is with hard-coded dot notation.

```
int k = x[indexOfInterest];     /* no problem */

double m = p.fieldOfInterest;   /* won't work */
double z = p[fieldOfInterest];  /* won't work */
/* No (sane) way to use field of interest */

double w = p.mass;              /* works fine */
```

The Java compiler does not treat text on either side of a dot as an expression, and thus it is not evaluated.

- See a compilers or programming languages class for more!

*: There is something called the “reflections” library which will let you access a field of a class using a String name, but it is not intended for casual use.

CC BY NC SA